GREENN SCHOOL · MÓDULO CSS

C3

CSS: Flexbox e Grid

Apostila do aluno · duração da aula: 2h

O que você vai aprender

- 1. Flexbox: o container
- 2. Flexbox: os filhos e o header
- 3. Grid: linhas e colunas
- 4. Grid: a página inteira com áreas

Antes de começar

Com o box model (C2) você controla cada caixa; com Flexbox e Grid você controla como as caixas se **organizam** entre si: lado a lado, centralizadas, em colunas. Regra de ouro das duas: quem manda no layout é sempre o **pai** (o container), não os filhos.

Prof. Walter Caciano · Greenn School

1 Flexbox: o container

`display: flex` no pai coloca os filhos **lado a lado** numa linha (o **eixo principal**). A partir daí, o pai decide como distribuir o espaço:

```
.linha {
  display: flex;
  gap: 12px;                        /* espaço entre os filhos */
  justify-content: space-between;   /* no eixo principal (horizontal) */
  align-items: center;              /* no eixo cruzado (vertical) */
  flex-direction: row;              /* row (padrão) ou column */
  flex-wrap: wrap;                  /* quebra a linha se não couber */
}
```

Propriedade (no pai)	Valores
`justify-content`	`flex-start`, `center`, `flex-end`, `space-between`, `space-around`, `space-evenly`
`align-items`	`stretch` (padrão), `flex-start`, `center`, `flex-end`
`flex-direction`	`row`, `column`, `row-reverse`, `column-reverse`
`flex-wrap`	`nowrap` (padrão: espreme), `wrap`
`gap`	o espaço entre filhos (vale para grid também)

> **Centralizar qualquer coisa** (a pergunta mais famosa do CSS): no pai, `display: flex; justify-content: center; align-items: center;` e uma altura. Pronto.

2 Flexbox: os filhos e o header

No filho, `flex: 1` diz "cresça para ocupar o que sobrar". Uma barra lateral de largura fixa e o conteúdo com `flex: 1` dão o layout de painel; dois filhos com `flex: 1` dividem igual; `flex: 2` pega o dobro.

```
.painel  { display: flex; gap: 12px; }
.lateral { width: 150px; }
.conteudo { flex: 1; }

/* o header clássico: logo de um lado, menu do outro */
.topo {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px 20px;
  background-color: #15171E;
}

.menu {
  display: flex;
  gap: 20px;
  list-style: none;   /* tira as bolinhas do ul */
  margin: 0;
  padding: 0;
}
```

3 Grid: linhas e colunas

Flex organiza em **uma** direção; Grid organiza em **duas** ao mesmo tempo. `grid-template-columns` define as colunas e os filhos vão preenchendo as células. A unidade `fr` é uma **fração** do espaço livre.

```
.grade {
  display: grid;
  grid-template-columns: repeat(3, 1fr);   /* 3 colunas iguais */
  gap: 12px;
}

/* outras formas de definir colunas */
.a { grid-template-columns: 200px 1fr; }       /* fixa + o resto */
.b { grid-template-columns: 1fr 2fr; }         /* a 2ª tem o dobro */
.c { grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); }
```

A última é a **galeria responsiva**: "quantas colunas de pelo menos 150px couberem". Estreitando a tela, o número de colunas cai sozinho, sem nenhuma media query.

4 Grid: a página inteira com áreas

```
.pagina {
  display: grid;
  grid-template-columns: 160px 1fr;
  grid-template-areas:
    "topo    topo"
    "lateral principal"
    "rodape  rodape";
  gap: 10px;
}

header { grid-area: topo; }
aside  { grid-area: lateral; }
main   { grid-area: principal; }
footer { grid-area: rodape; }
```

Cada string de `grid-template-areas` é uma linha e cada palavra uma coluna; repetir o nome estica a área por várias células. Cada filho diz qual área ocupa com `grid-area`. É o jeito mais **legível** de montar um layout, e trocar o desenho no celular é só reescrever as strings dentro de uma media query (próxima aula).

Quando usar	Ferramenta
Itens numa linha ou coluna: menu, botões, cartão com ícone e texto, centralizar	**Flexbox**
Grade de cartões, galeria, o layout geral da página	**Grid**
Os dois juntos	Grid para a página, flex dentro de cada bloco (é o normal)

5 Exercícios

Do 1 ao 5, Flexbox (linha, distribuição, centralizar, flex: 1 e o header); 6 e 7, Grid; o 8 é uma página inteira com áreas. Em cada um, compare a sua saída com a esperada — cada letra, espaço e acento conta.

01 Três caixas em linha — NÍVEL 1

.linha com display flex e gap 16px.

Saída esperada: Caixas lado a lado

02 Um em cada ponta — NÍVEL 1

.barra com flex e justify-content space-between.

Saída esperada: Logo à esquerda, Sair à direita

03 No centro de tudo — NÍVEL 2

.hero com 200px de altura, fundo escuro, flex, justify e align center.

Saída esperada: Caixa no meio exato

04 Menu fixo, conteúdo que cresce — NÍVEL 2

.layout flex com gap; .lateral width 150px; .conteudo flex: 1.

Saída esperada: Painel com menu lateral

05 Cabeçalho com menu — NÍVEL 2

.topo flex space-between center escuro; .menu flex sem bolinhas; links brancos.

Saída esperada: Header de site

06 Grade de três colunas — NÍVEL 2

.grade com grid, repeat(3, 1fr) e gap 12px.

Saída esperada: Duas linhas de três

07 Galeria que se adapta — NÍVEL 2

.galeria com repeat(auto-fit, minmax(150px, 1fr)).

Saída esperada: Colunas que se ajustam

08 Página inteira com grid — DESAFIO

.pagina com colunas 160px 1fr e áreas topo/lateral/principal/rodape; cada bloco com seu grid-area.

Saída esperada: Header, menu, conteúdo e rodapé

> **Travou?** Volte ao exemplo da seção correspondente, rode o código no site e mude um valor por vez para entender o que cada parte faz.